

Introduction to Python Programming

Chapter 9: Exceptions and Error Handling

Introduction

You've probably encountered errors in your programs - your code crashes with red text and stops running. These are **exceptions**. Until now, exceptions meant your program failed. But professional programmers expect errors and handle them gracefully. This chapter teaches you how to catch exceptions, handle them properly, and keep your programs running even when things go wrong.

1. Understanding Exceptions

What is an Exception?

An **exception** is an error that occurs during program execution. When Python encounters a problem it can't handle, it "raises" (throws) an exception.

```
# This causes an exception
number = int("hello") # ValueError: invalid literal for int()
```

What happens without exception handling:

1. Python encounters an error
2. Raises an exception
3. Prints error message
4. **Program crashes and stops**

Common Exception Types

Python has many built-in exception types. Here are the most common:

ValueError

Raised when a function gets the right type but wrong value:

```
number = int("abc") # ValueError: invalid literal for int()
```

TypeError

Raised when an operation is performed on the wrong type:

```
result = "hello" + 5 # TypeError: can only concatenate str to str
```

ZeroDivisionError

Raised when dividing by zero:

```
result = 10 / 0 # ZeroDivisionError: division by zero
```

IndexError

Raised when accessing an invalid list index:

```
numbers = [1, 2, 3]
print(numbers[5]) # IndexError: list index out of range
```

KeyError

Raised when accessing a non-existent dictionary key:

```
person = {"name": "Alice"}
print(person["age"]) # KeyError: 'age'
```

FileNotFoundError

Raised when trying to open a file that doesn't exist:

```
file = open("nonexistent.txt") # FileNotFoundError
```

NameError

Raised when using a variable that doesn't exist:

```
print(undefined_variable) # NameError: name 'undefined_variable' is not defined
```

AttributeError

Raised when accessing a non-existent attribute:

```
text = "hello"
text.append("!") # AttributeError: 'str' object has no attribute 'append'
```

2. Try-Except Blocks

Try-except blocks let you handle exceptions gracefully instead of crashing.

Basic Try-Except

```
try:
    # Code that might cause an exception
    number = int(input("Enter a number: "))
    print(f"You entered: {number}")
except:
    # Code that runs if ANY exception occurs
    print("That's not a valid number!")
```

How it works:

1. Python tries to run code in the `try` block
2. If an exception occurs, jump to `except` block
3. Program continues running (doesn't crash!)

Catching Specific Exceptions

It's better to catch specific exception types:

```
try:
    number = int(input("Enter a number: "))
    result = 10 / number
    print(f"Result: {result}")
except ValueError:
    print("That's not a valid number!")
except ZeroDivisionError:
    print("Cannot divide by zero!")
```

Why catch specific exceptions?

- Different errors need different handling
- More precise error messages
- Unexpected errors still crash (which is good for debugging!)

Multiple Exceptions in One Except

```
try:
    number = int(input("Enter a number: "))
    result = 10 / number
except (ValueError, ZeroDivisionError):
    print("Invalid input or cannot divide by zero!")
```

3. The Else Clause

The `else` clause runs if NO exception occurred:

```
try:
    number = int(input("Enter a number: "))
except ValueError:
    print("Invalid number!")
else:
    # Only runs if no exception
    print(f"Success! You entered {number}")
```

When to use `else`:

- Code that should only run if `try` succeeds
- Keeps `try` block minimal (best practice)

```
try:
    age = int(input("Enter your age: "))
except ValueError:
    print("Invalid age!")
else:
    if age >= 18:
        print("You are an adult")
    else:
        print("You are a minor")
```

4. The Finally Clause

The `finally` clause **always** runs, whether an exception occurred or not:

```
try:
    number = int(input("Enter a number: "))
    result = 10 / number
    print(f"Result: {result}")
except ValueError:
    print("Invalid number!")
except ZeroDivisionError:
    print("Cannot divide by zero!")
finally:
    print("This ALWAYS runs!")
```

When to use `finally`:

- Cleanup code (close files, release resources)
- Code that must run no matter what
- Logging or tracking

```
file = None
try:
    file = open("data.txt", "r")
    content = file.read()
    print(content)
except FileNotFoundError:
    print("File not found!")
finally:
    if file:
        file.close() # Always close the file
    print("Cleanup complete")
```

5. Getting Exception Details

Accessing the Exception Object

```
try:
    number = int(input("Enter a number: "))
except ValueError as e:
    print(f"Error occurred: {e}")
    print(f"Error type: {type(e).__name__}")
```

Output if user enters "abc":

```
Error occurred: invalid literal for int() with base 10: 'abc'
Error type: ValueError
```

Practical Example: Safe Input

```
def get_integer(prompt):
    """Get integer input with error handling."""
    while True:
        try:
            value = int(input(prompt))
            return value
        except ValueError as e:
            print(f"Invalid input: {e}")
            print("Please enter a valid integer.")

# Use the function
age = get_integer("Enter your age: ")
print(f"You are {age} years old")
```

6. Raising Exceptions

You can **raise** (throw) your own exceptions:

Basic Raise

```
def set_age(age):
    if age < 0:
        raise ValueError("Age cannot be negative!")
    if age > 150:
        raise ValueError("Age seems unrealistic!")
    return age

try:
    set_age(-5)
except ValueError as e:
    print(f"Error: {e}")
```

Re-raising Exceptions

Sometimes you want to handle an exception but still let it propagate:

```
try:
    number = int(input("Enter number: "))
except ValueError as e:
    print("Logging error...") # Do some handling
    raise # Re-raise the same exception
```

Custom Exception Messages

```
def divide(a, b):
    if b == 0:
        raise ZeroDivisionError("Custom message: Division by zero is not allowed!")
    return a / b

try:
    result = divide(10, 0)
except ZeroDivisionError as e:
    print(e) # Custom message: Division by zero is not allowed!
```

7. Exception Hierarchy

Python exceptions are organized in a hierarchy. Catching a parent exception catches all child exceptions.

The Exception Family Tree

```
BaseException
|
+-- SystemExit
+-- KeyboardInterrupt
+-- Exception
    |
    +-- ArithmeticError
    |     |
    |     +-- ZeroDivisionError
    |     +-- OverflowError
    |     +-- FloatingPointError
    |
    +-- LookupError
    |     |
    |     +-- IndexError
    |     +-- KeyError
    |
    +-- ValueError
    +-- TypeError
    +-- NameError
    +-- AttributeError
    +-- ImportError
    +-- OSError
    |     |
    |     +-- FileNotFoundError
    |     +-- PermissionError
    |
    +-- RuntimeError
    +-- (many more...)
```

Understanding the Hierarchy

BaseException is the root of all exceptions:

- `SystemExit` - raised by `sys.exit()`
- `KeyboardInterrupt` - raised when user presses Ctrl+C
- `Exception` - base for most user-defined and built-in exceptions

Exception is what you usually catch:

- `ArithmeticError` - parent of math-related errors
 - `ZeroDivisionError` - dividing by zero
 - `OverflowError` - result too large
- `LookupError` - parent of indexing/key errors
 - `IndexError` - invalid list/tuple index
 - `KeyError` - invalid dictionary key
- `ValueError` - right type, wrong value
- `TypeError` - wrong type for operation
- `OSError` - parent of file/system errors
 - `FileNotFoundError` - file doesn't exist
 - `PermissionError` - insufficient permissions

Catching Multiple Related Exceptions

```
try:
    numbers = [1, 2, 3]
    index = int(input("Enter index: "))
    print(numbers[index])
except LookupError: # Catches both IndexError and KeyError
    print("Invalid index or key!")
except ValueError:
    print("Not a valid number!")
```

Catching Parent Exceptions

```
try:
    # Some risky operation
    result = 10 / 0
except ArithmeticError: # Catches ZeroDivisionError, OverflowError, etc.
    print("Math error occurred!")
```


Order Matters!

Catch specific exceptions before general ones:

```
# CORRECT - specific first
try:
    # code
except ValueError:
    print("Value error")
except Exception:
    print("Some other error")

# WRONG - general first catches everything
try:
    # code
except Exception: # This catches EVERYTHING
    print("Some error")
except ValueError: # Never reached!
    print("Value error")
```

Why order matters:

```
try:
    number = int("abc")
except Exception: # Catches ValueError (and everything else)
    print("Caught by Exception")
except ValueError: # Never executed
    print("Caught by ValueError")
```

8. Preview: Context Managers and the `with` Statement

The Resource Management Problem

When working with resources (files, database connections, network sockets, locks), you must ensure they're properly released, even when exceptions occur:

```
# The problem: resource might not be released
file = open("data.txt", "r")
content = file.read() # If this raises an exception...
file.close()          # ...this never runs!
```

If an exception occurs, the file stays open, wasting system resources.

The Traditional Solution: Try-Finally

```
file = None
try:
    file = open("data.txt", "r")
    content = file.read()
    # Process content
except FileNotFoundError:
    print("File not found!")
finally:
    if file:
        file.close() # Always runs
```

This works, but it's verbose and easy to forget.

The Better Solution: Context Managers

Python's `with` statement uses **context managers** to guarantee cleanup:

```
with open("data.txt", "r") as file:
    content = file.read()
    # Process content
# File automatically closes here, even if exception occurs
```

How it works:

1. `with` statement enters a context
2. Resource is acquired (file opened)
3. Your code runs in the indented block
4. Context manager ensures cleanup (file closed), even if exception occurs
5. Exception propagates normally after cleanup

Why Context Managers Matter

Not just for files! Context managers work with any resource:

```
# Database connections (you'll learn this later)
with database.connect() as conn:
    conn.execute(query)
# Connection automatically closed

# Thread locks (for multi-threading)
with lock:
    # Critical section
    shared_resource += 1
# Lock automatically released
```

```
# Network sockets
with socket.create_connection(address) as sock:
    sock.sendall(data)
# Socket automatically closed
```

Creating Your Own Context Manager

You can create custom context managers using a class:

```
class FileLogger:
    def __init__(self, filename):
        self.filename = filename
        self.file = None

    def __enter__(self):
        """Called when entering 'with' block."""
        print(f"Opening {self.filename}")
        self.file = open(self.filename, "a")
        return self.file

    def __exit__(self, exc_type, exc_value, traceback):
        """Called when exiting 'with' block (even if exception)."""
        print(f"Closing {self.filename}")
        if self.file:
            self.file.close()
        # Return False to let exceptions propagate
        return False

# Use it
with FileLogger("app.log") as log:
    log.write("Application started\n")
# Even if exception occurs here, file closes
```

The `__exit__` method:

- Always runs (like `finally`)
- Receives exception info if one occurred
- Can suppress exceptions by returning `True` (rare)

Simple Example: Timer Context Manager

```
import time

class Timer:
    def __enter__(self):
        self.start = time.time()
```

```

        return self

    def __exit__(self, exc_type, exc_value, traceback):
        self.end = time.time()
        print(f"Elapsed time: {self.end - self.start:.2f} seconds")
        return False

# Use it
with Timer():
    # Some time-consuming operation
    total = sum(range(1000000))
# Automatically prints elapsed time

```

Context Managers with contextlib

Python provides `contextlib` for easier context manager creation:

```

from contextlib import contextmanager

@contextmanager
def temporary_value(var, temp_value):
    """Temporarily change a variable's value."""
    original = var
    try:
        yield temp_value # This is what 'as' captures
    finally:
        # Cleanup: restore original (this always runs)
        print(f"Restored to {original}")

# Can't directly demo with immutable types, but concept:
# with temporary_value(my_list, []) as temp:
#     # Work with temp
#     pass
# my_list is restored

```

Key Concepts

Context managers guarantee:

1. **Setup code runs** when entering context
2. **Cleanup code runs** when exiting context
3. **Cleanup happens even with exceptions**
4. **Exceptions still propagate** after cleanup

When to use context managers:

- Opening files
- Database connections
- Network sockets
- Thread locks
- Temporary state changes
- Timing operations
- Any resource that needs cleanup

Looking Ahead

In the next chapter on File I/O, you'll see context managers in action:

```
# This pattern guarantees file closure
with open("data.txt", "r") as file:
    content = file.read()
# File automatically closed, even if exception occurs
```

Remember: The `with` statement isn't about files—it's about **automatic resource management with exception safety**. Files are just the most common use case!

9. Practical Examples

Example 1: Safe Division Calculator

```
def safe_divide():
    """Calculator that handles all division errors."""
    try:
        num1 = float(input("Enter first number: "))
        num2 = float(input("Enter second number: "))
        result = num1 / num2
    except ValueError:
        print("Error: Please enter valid numbers!")
        return None
    except ZeroDivisionError:
        print("Error: Cannot divide by zero!")
        return None
    else:
        print(f"Result: {result}")
        return result
    finally:
        print("Calculation attempt complete.")
```

```
safe_divide()
```

Example 2: List Access with Error Handling

```
def get_element(lst, index):
    """Safely get element from list."""
    try:
        return lst[index]
    except IndexError:
        print(f"Error: Index {index} out of range (list has {len(lst)} items)")
        return None
    except TypeError:
        print("Error: Index must be an integer!")
        return None

numbers = [10, 20, 30, 40, 50]
print(get_element(numbers, 2))    # 30
print(get_element(numbers, 10))   # None (with error message)
print(get_element(numbers, "a"))  # None (with error message)
```

Example 3: Dictionary Access with Default

```
def get_config(config_dict, key, default=None):
    """Get config value with error handling."""
    try:
        return config_dict[key]
    except KeyError:
        print(f"Warning: '{key}' not found, using default: {default}")
        return default
    except TypeError:
        print("Error: First argument must be a dictionary!")
        return None

config = {"theme": "dark", "language": "en"}
print(get_config(config, "theme"))    # dark
print(get_config(config, "font", "Arial")) # Arial (uses default)
```

Example 4: Input Validation Loop

```
def get_positive_integer(prompt):
    """Get positive integer with validation."""
    while True:
        try:
            value = int(input(prompt))
            if value <= 0:
```

```

        raise ValueError("Number must be positive!")
    return value
except ValueError as e:
    if "invalid literal" in str(e):
        print("Error: Please enter a valid integer!")
    else:
        print(f"Error: {e}")

age = get_positive_integer("Enter your age: ")
print(f"Your age: {age}")

```

Example 5: Multiple Operations with Comprehensive Error Handling

```

def process_data(data_list, index, divisor):
    """Process data with multiple error checks."""
    try:
        # Check if list is empty
        if not data_list:
            raise ValueError("Data list is empty!")

        # Get element
        value = data_list[index]

        # Convert to number if string
        if isinstance(value, str):
            value = float(value)

        # Perform calculation
        result = value / divisor

    except IndexError:
        print(f"Error: Index {index} out of range!")
        return None
    except ValueError as e:
        print(f"Error: {e}")
        return None
    except ZeroDivisionError:
        print("Error: Cannot divide by zero!")
        return None
    except TypeError:
        print("Error: Invalid data type!")
        return None
    else:
        print(f"Success: Result is {result}")
        return result
    finally:
        print("Processing complete.")

```

```
# Test cases
data = [10, "20", 30, 40]
process_data(data, 1, 2) # Success
process_data(data, 10, 2) # IndexError
process_data(data, 1, 0) # ZeroDivisionError
```

10. Best Practices

Do's

✓ Catch specific exceptions:

```
# Good
except ValueError:
    ...
```

✓ Keep try blocks small:

```
# Good - only risky code in try
try:
    number = int(user_input)
except ValueError:
    ...

# Process the number here (outside try)
result = number * 2
```

✓ Provide helpful error messages:

```
except ValueError as e:
    print(f"Invalid input: {e}")
    print("Please enter a number between 1 and 100")
```

✓ Use finally for cleanup:

```
try:
    file = open("data.txt")
    # process file
finally:
    file.close()
```

Don'ts

✗ Don't use bare except:


```
# Bad - catches everything, even Ctrl+C!
try:
    ...
except:
    ...
```

✗ Don't hide errors silently:

```
# Bad - errors disappear without trace
try:
    risky_operation()
except:
    pass # Silent failure!
```

✗ Don't catch exceptions you can't handle:

```
# Bad - what will you do with this?
try:
    ...
except Exception as e:
    print("Something went wrong") # Too vague!
```

✗ Don't use exceptions for control flow:

```
# Bad - exceptions shouldn't replace if statements
try:
    value = my_dict[key]
except KeyError:
    value = default

# Good - use .get() instead
value = my_dict.get(key, default)
```

11. When to Use Exception Handling

Use Exception Handling When:

1. User input is involved:

```
try:
    age = int(input("Enter age: "))
except ValueError:
    print("Invalid age!")
```

2. File operations:

```
try:
    with open("file.txt") as f:
        content = f.read()
except FileNotFoundError:
    print("File not found!")
```

3. Network operations:

```
try:
    response = requests.get(url)
except requests.ConnectionError:
    print("Network error!")
```

4. Type conversions:

```
try:
    number = float(value)
except ValueError:
    print("Cannot convert to number!")
```

Don't Use Exception Handling When:

1. You can check beforehand:

```
# Bad
try:
    value = my_list[5]
except IndexError:
    ...

# Good
if len(my_list) > 5:
    value = my_list[5]
```

2. Logic errors (bugs):

```
# Bad - this is a bug, not an exception to handle
try:
    result = calculat_sum(a, b) # Typo in function name
except NameError:
    ...
```

Key Takeaways

- ✓ **Exceptions** are errors that occur during execution
 - ✓ **Try-except** blocks handle exceptions gracefully
 - ✓ **Catch specific exceptions** for better error handling
 - ✓ **else** runs if no exception occurred
 - ✓ **finally** always runs (cleanup code)
 - ✓ **raise** throws exceptions manually
 - ✓ **Exception hierarchy** - parent exceptions catch children
 - ✓ **Order matters** - specific exceptions before general ones
 - ✓ **Keep try blocks small** - only risky code
 - ✓ **Provide helpful messages** - tell users what went wrong
 - ✓ **Don't hide errors** - log or handle them properly
-

Reflection Questions

1. What's the difference between try and except blocks?
 2. When should you use else vs finally?
 3. Why catch specific exceptions instead of using bare except?
 4. How do you access the exception message?
 5. What's the purpose of raising exceptions?
 6. Why does order matter when catching multiple exceptions?
 7. When should you use exception handling vs checking conditions?
 8. What happens if you don't handle an exception?
-

Practice Exercises

1. **Safe Input:** Create a function that gets a number from user and keeps asking until valid
2. **List Safe Access:** Write a function that safely accesses list elements by index
3. **Calculator:** Build a calculator that handles all possible errors (division by zero, invalid input, etc.)
4. **Dictionary Lookup:** Create a function that safely gets dictionary values with custom error messages
5. **File Reader:** Write code that tries to read a file and handles FileNotFoundError

6. **Age Validator:** Get user's age and validate it's between 0 and 120
7. **Multiple Operations:** Create a function that performs multiple operations and handles all possible exceptions
8. **Custom Exception:** Define a custom exception for negative numbers and use it
9. **Error Logger:** Write a function that catches exceptions and logs them to a file
10. **Robust Program:** Combine everything - build a program that takes user input, processes it, and handles all errors gracefully